

ML course season 2

Лекция 3: Метод опорных векторов (SVM)

Паша Кашмель

Курс по машинному обучению

6 апреля 2026 г.

План лекции

- 1 Идея максимизации зазора
- 2 Hard Margin и Soft Margin. Параметр C
- 3 Ядровой трюк
- 4 Масштабирование признаков
- 5 Настройка SVM: гиперпараметры

Задача линейной классификации: что мы ищем?

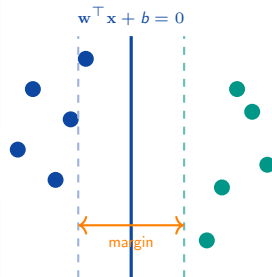
Проблема

Через обучающую выборку можно провести **бесконечно много** разделяющих гиперплоскостей.

Какую выбрать?

Идея SVM

Выбираем гиперплоскость с **максимальным зазором** (margin) — максимальным расстоянием до ближайших точек обоих классов.



Разделяющая гиперплоскость и margin

Гиперплоскость задаётся уравнением:

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

Опорные векторы — точки, ближайшие к гиперплоскости:

$$\mathbf{w}^\top \mathbf{x}_i + b = +1 \quad (\text{класс } +1)$$

$$\mathbf{w}^\top \mathbf{x}_i + b = -1 \quad (\text{класс } -1)$$

Ширина зазора

$$\text{margin} = \frac{2}{\|\mathbf{w}\|}$$

Задача оптимизации

Максимизировать $\frac{2}{\|\mathbf{w}\|}$

$\Leftrightarrow$ Минимизировать $\frac{1}{2} \|\mathbf{w}\|^2$

при ограничениях:

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$$

для всех $i = 1, \dots, n$.

Почему максимальный зазор лучше?

Интуиция:

- Большой зазор $\Rightarrow$ модель менее чувствительна к шуму в данных
- Новые точки «уверенно» классифицируются
- Теоретически: VC-размерность контролируется через margin

Только опорные векторы важны

Предсказание зависит только от **опорных векторов** — точек на краях зазора.

Остальные точки можно убрать, модель не изменится.

Сравнение с логрегрессией

Логрегрессия	SVM
Любая граница	Макс. margin
Учит все точки	Только SV
Вероятности	Расстояние
Быстро	Медленнее

Hard Margin SVM: ограничения

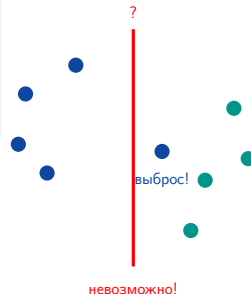
Hard Margin

Требуем, чтобы **все** точки были классифицированы верно и лежали за пределами зазора:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i$$

Проблемы:

- Не работает, если данные **линейно неразделимы**
- Чрезмерно чувствителен к **выбросам**
- В реальных задачах почти никогда не применяется



Soft Margin SVM: допускаем ошибки

Вводим **переменные ослабления** $\xi_i \geq 0$ —
насколько точка нарушает ограничение:

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i$$

Задача Soft Margin SVM

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

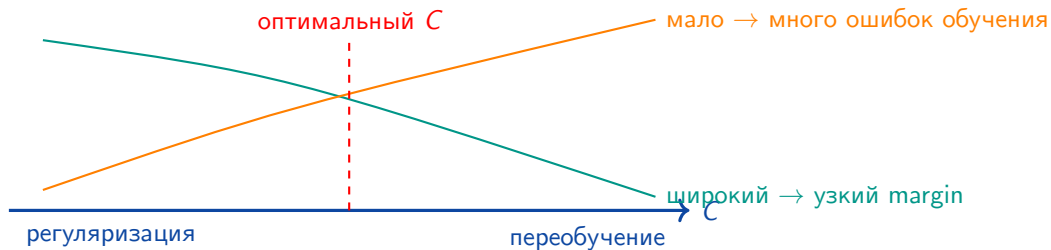
$$\text{при } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Параметр C

C — **штраф за ошибку**:

Большой C	$\approx$ Hard Margin Меньше ошибок Уже зазор
Малый C	Широкий зазор Больше ошибок Лучше обобщение

Компромисс: ошибки vs зазор



Выбор C — задача подбора гиперпараметра

Используем кросс-валидацию или GridSearchCV.

Когда данные нелинейно разделимы

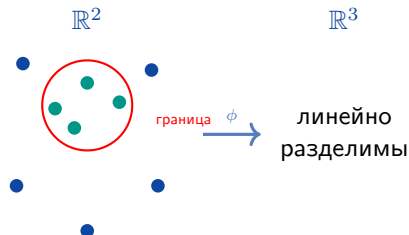
Проблема:

- В исходном пространстве классы могут не разделяться прямой линией
- Пример: «XOR»-задача, вложенные круги

Идея: отобразить данные в **пространство большей размерности** $\phi(\mathbf{x})$, где они разделимы линейно.

Пример

$$\mathbf{x} = (x_1, x_2) \rightarrow \phi(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)$$



Ядровой трюк: вычисляем скалярные произведения

Ключевое наблюдение

SVM при обучении и предсказании использует только **скалярные произведения** $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$.

Ядровая функция:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$$

Вычисляем результат скалярного произведения в ϕ -пространстве **без явного перехода** туда!

Kernel trick

Заменяем $\mathbf{x}_i^\top \mathbf{x}_j$ на $K(\mathbf{x}_i, \mathbf{x}_j)$ — получаем нелинейный SVM.

Популярные ядра:

Линейное: $\mathbf{x}_i^\top \mathbf{x}_j$

Полиномиальное: $(\gamma \mathbf{x}_i^\top \mathbf{x}_j + r)^d$

RBF (Гауссово): $\exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$

Сигмоидальное: $\tanh(\gamma \mathbf{x}_i^\top \mathbf{x}_j + r)$

RBF ядро: интуиция и параметр gamma

RBF (Radial Basis Function):

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

- γ контролирует «радиус влияния» каждой точки
- RBF соответствует ϕ в **бесконечномерном** пространстве

Влияние γ

Большой γ	Узкие «холмы» → переобучение
Малый γ	Гладкая граница → недообучение

Когда какое ядро?

Линейное:

Много признаков ($d \gg n$), текст, разреженные данные

RBF:

Нелинейные зависимости, $n \gg d$, таблицы

Poly:

Изображения, NLP (реже), когда важен порядок взаимодействий

SVM чувствителен к масштабу!

Почему важен масштаб?

Расстояние до гиперплоскости зависит от $\|\mathbf{w}\|$. Если признак $x_1 \in [0, 1]$, а $x_2 \in [0, 1000]$, то x_2 будет доминировать.

Результат без нормализации

- Неверная граница решений
- Неоптимальный margin
- Медленная сходимость

Обязательно делаем:

$$\text{StandardScaler} \quad \frac{x - \mu}{\sigma}$$

$$\text{MinMaxScaler} \quad \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing \
    import StandardScaler
from sklearn.svm import SVC

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(kernel='rbf'))
])
```

Основные гиперпараметры SVM

Параметр	Значения	Что делает
C	$10^{-3} \dots 10^3$	Штраф за нарушение margin; больше C — меньше ошибок на train
kernel	linear, rbf, poly	Вид ядра; rbf — лучший default
gamma	scale, auto, число	Для RBF/poly: ширина ядра; больше — сложнее граница
degree	2, 3, 4, ...	Степень полиномиального ядра
class_weight	balanced, dict	Веса классов при дисбалансе

Рекомендации по умолчанию

```
SVC(kernel='rbf', C=1.0, gamma='scale', class_weight='balanced')
```

Подбор параметров через GridSearchCV

```
from sklearn.model_selection \
    import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing \
    import StandardScaler
from sklearn.svm import SVC

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(probability=True))
])

param_grid = {
    'svm__C': [0.1, 1, 10, 100],
    'svm__kernel': ['linear', 'rbf'],
    'svm__gamma': ['scale', 0.1, 0.01],
}
```

Почему Pipeline?

- 1 Scaler обучается только на train
- 2 Нет утечки данных при CV
- 3 Один объект = полный пайплайн

class_weight

При дисбалансе классов:

```
SVC(class_weight='balanced')
```

Автоматически взвешивает классы

SVM: достоинства и ограничения

Достоинства

- Эффективен при **малом числе примеров**
- Работает в **высоких размерностях** (с linear kernel)
- Kernel trick — огромная гибкость
- Чётко определённая задача оптимизации

Ограничения

- Медленно на **больших выборках** ($n > 10^5$)
- Необходимо масштабировать признаки
- Подбор C, γ — трудоёмко
- Плохо интерпретирует вероятности
- Сложная работа с многоклассовыми задачами

	Линейный SVM	RBF SVM
Применение	$d \gg n$, текст	табличные данные
Сложность	$O(nd)$	$O(n^2d)$
Использует	Решение	Оптимизация

Что мы разобрали

- 1 **Margin** — зазор и опорные векторы
- 2 **Hard/Soft Margin** — параметр C
- 3 **Kernel trick** — нелинейные границы
- 4 **RBF и poly** — параметр γ
- 5 **Масштабирование** — обязательно!
- 6 **GridSearchCV** — подбор гиперпараметров

Домашнее задание

- 1 Изучить раздел 5.3 учебника Яндекса по ML
- 2 Посмотреть видео-лекцию по SVM
- 3 **Выполнить семинарский ноутбук:** реализовать LinearSVM, подобрать параметры, визуализировать границу решений

Вопросы?

Спасибо за внимание!

Полезные ресурсы

`education.yandex.ru/handbook/ml` • `scikit-learn.org`
`youtube.com/playlist?list=PLA0M1Bcd0w8zxDIDOTQHsX68MCDOAJDtj`